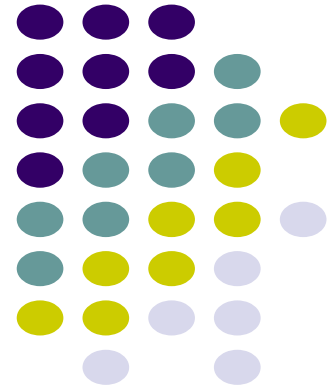


Redes de Computadores

Prof. Flavio Barbieri Gonzaga

flavio.gonzaga@unifal-mg.edu.br

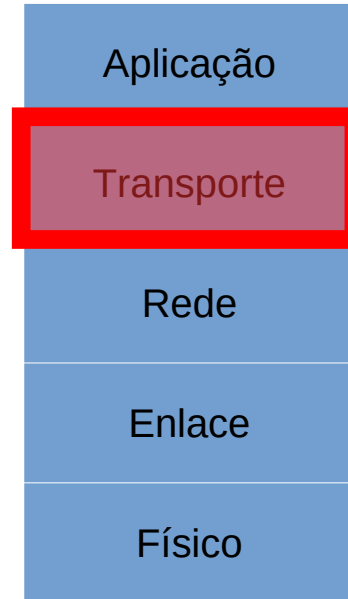
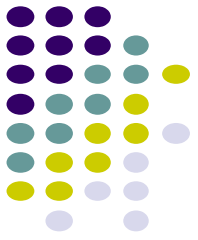




Sumário

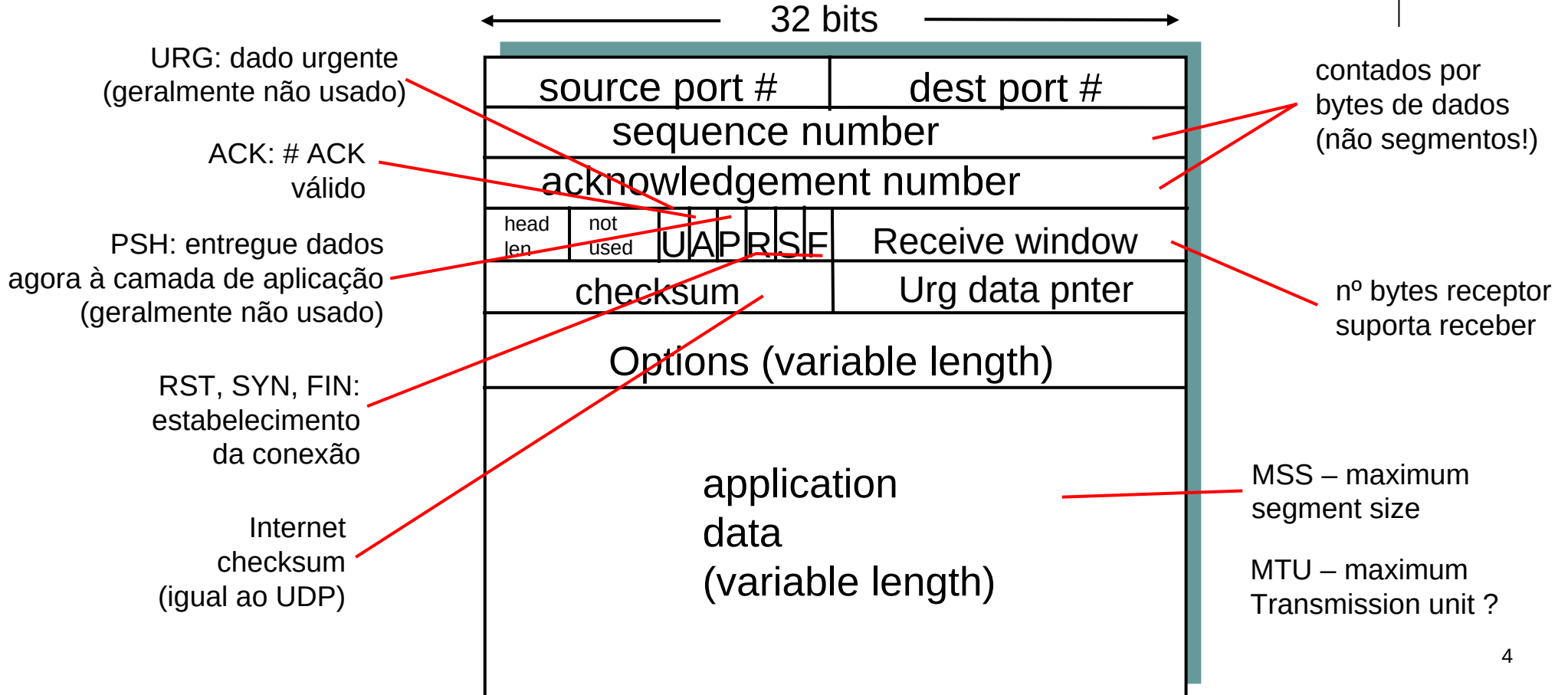
- TCP
 - Estrutura do segmento
 - Número de sequência e ACK
 - TCP Round Trip Time e Timeout
 - Duplicação do tempo de Timeout
 - Retransmissão Rápida
 - Controle de Fluxo

Camada de Transporte



Pilha de protocolos da Internet
(cinco camadas)

Estrutura Segmento





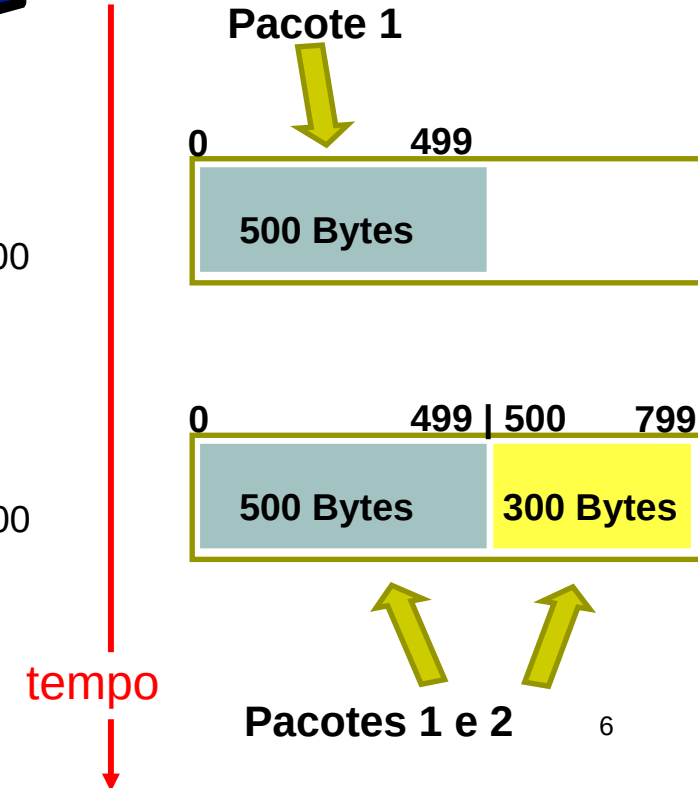
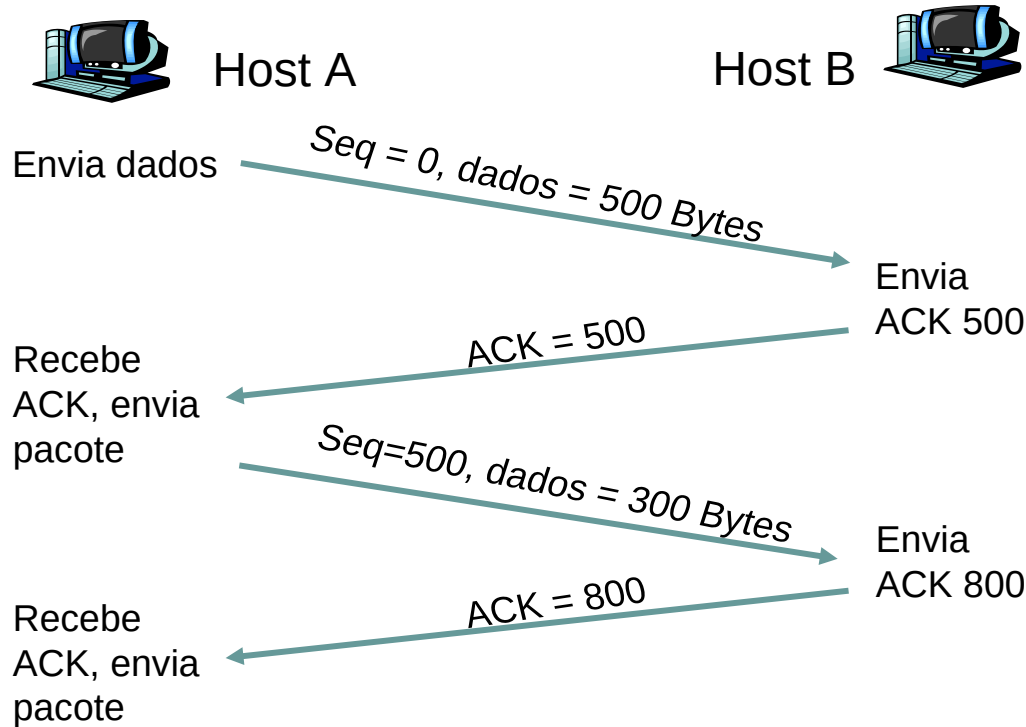
Número de sequência e ACK

- Número de sequência se refere ao primeiro byte do segmento
- Número do ACK se refere ao número de sequência do próximo pacote que o receptor espera receber

Número de sequência e ACK



Exemplo

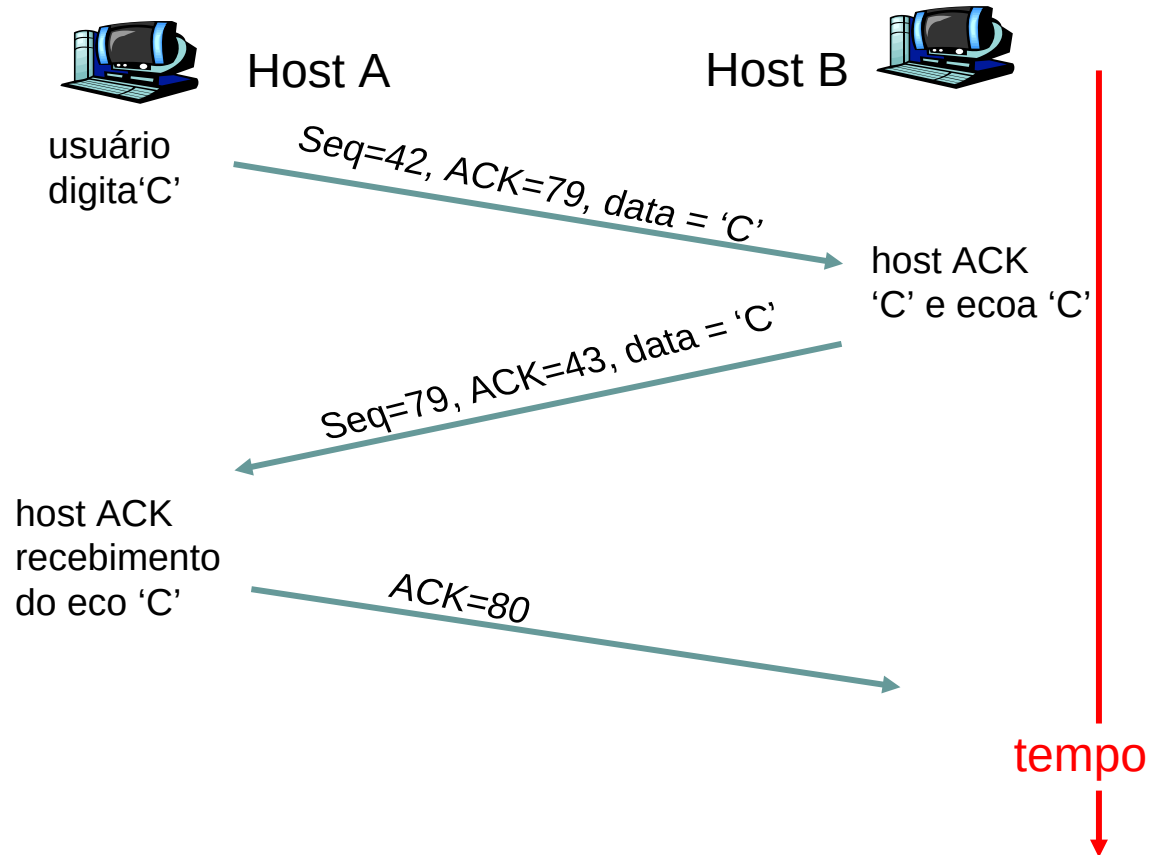


Número de sequência e ACK



Exemplo telnet

**Transmissão
Full Duplex!
Dados e ACK no
mesmo pacote.**





TCP Round Trip Time e Timeout

Q: Como definir um valor de timeout?

- Maior do que o RTT
 - (Mas RTT varia!)
- Se for muito pequeno:
 - Retransmissões desnecessárias
- Muito longo:
 - Reação demorada à perda de pacotes

Q: Como estimar o RTT?

- **SampleRTT**: tempo medido da transmissão do segmento TCP até o recebimento do ACK
 - Feito para pacotes que não seja retransmissão!
- **SampleRTT** varia!
 - Uma boa abordagem é estimar o timeout considerando uma média de SampleRTT's

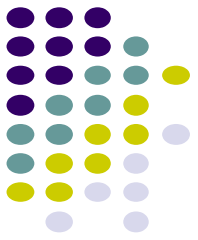


TCP Round Trip Time e Timeout

$$\text{EstimatedRTT} = (1 - \alpha) * \text{EstimatedRTT} + \alpha * \text{SampleRTT}$$

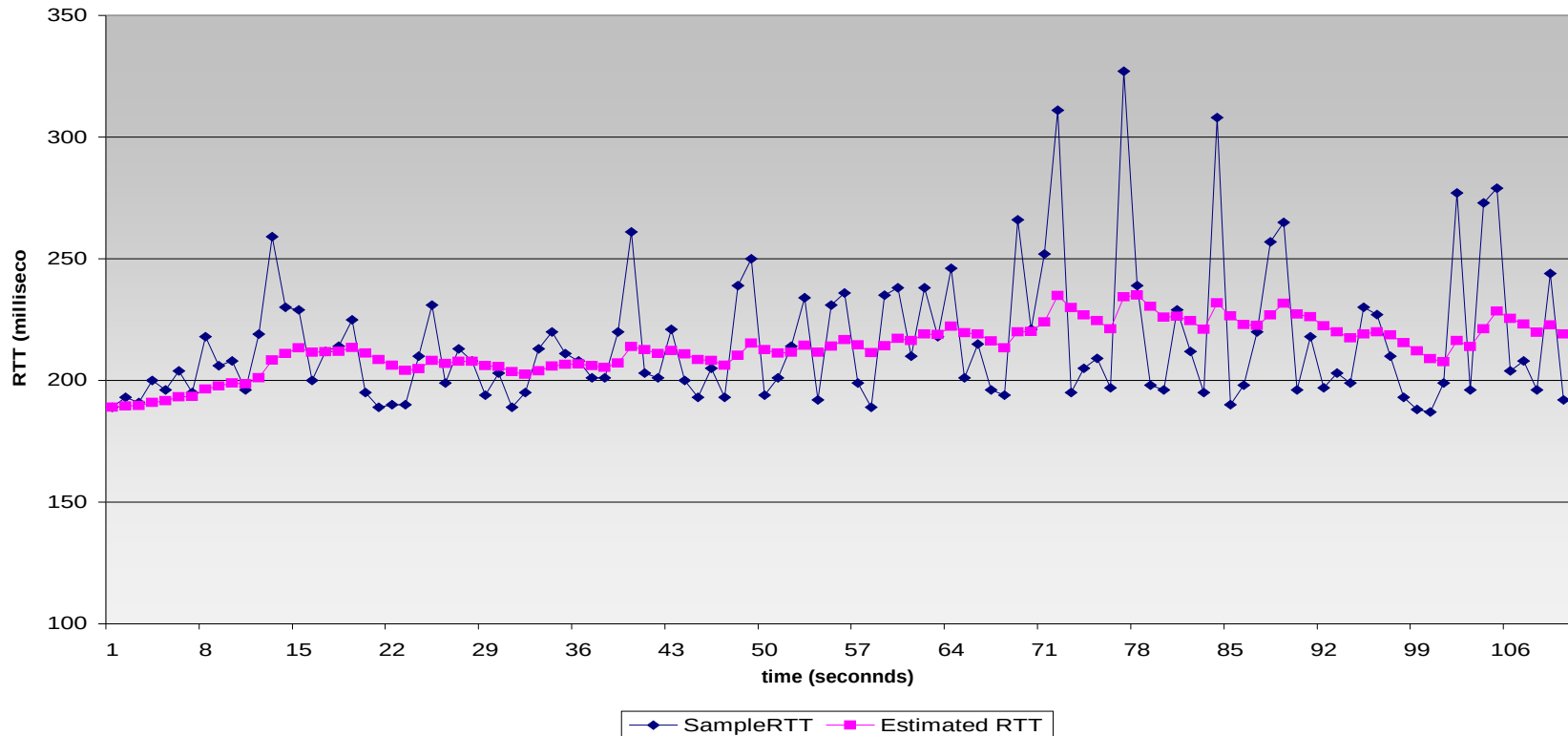
- Influência de medições passadas decrescem rápido!
- Valor típico: $\alpha = 0.125$

TCP Round Trip Time e Timeout



Exemplo estimativa RTT

RTT: gaia.cs.umass.edu to fantasia.eurecom.fr





TCP Round Trip Time e Timeout

Variação do RTT

- **EstimatedRTT** mais uma margem de segurança
- Variação do Sample RTT em relação ao EstimatedRTT:

$$\text{DevRTT} = (1-\beta) * \text{DevRTT} + \beta * |\text{SampleRTT} - \text{EstimatedRTT}|$$

(geralmente, $\beta = 0.25$)

Obtendo o timeout:

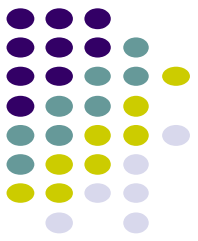
$$\text{TimeoutInterval} = \text{EstimatedRTT} + 4 * \text{DevRTT}$$

Duplicação do tempo de *timeout*



- Sempre que o *timeout* de um pacote se esgota:
 - O pacote retransmitido terá um valor de *timeout* que será o dobro do valor anterior (do *timeout* esgotado)
- Exemplo:
 - Um pacote cujo *timeout* era de 0,5 segundo e foi esgotado, terá a sua retransmissão com um *timeout* igual a 1,0 segundo
 - Crescimento exponencial

Duplicação do tempo de *timeout*



- Atraso ou perda indica congestionamento na rede!
- Ao duplicar o valor de *timeout* para retransmissões, a taxa de envio de pacotes está sendo reduzida, contribuindo para que a rede se “descongestione”





Retransmissão Rápida

- Um dos problemas em se retransmitir quando o *timeout* expira se deve ao fato de que o tempo de *timeout* pode ser longo demais, causando um atraso considerável fim-a-fim



Retransmissão Rápida

- O TCP possui um mecanismo de retransmissão rápida, que busca reduzir o atraso fim-a-fim
- Ao se detectar o 3º ACK duplicado de um mesmo pacote, o pacote que o ACK requisita será retransmitido



Retransmissão Rápida

Exemplo



Host A



Host B

Envia dados

Recebe ACK

1º ACK duplicado

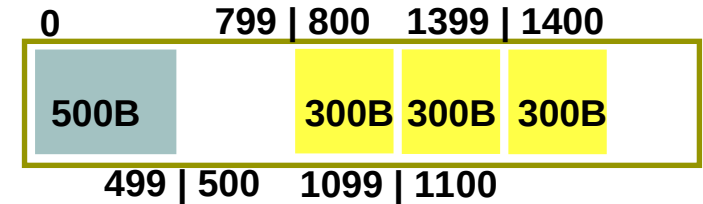
2º ACK duplicado

3º ACK duplicado

timeout



Envia
ACK 500

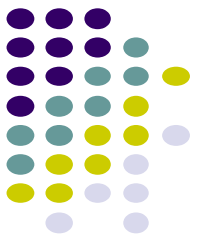




Controle de Fluxo

- Mecanismo que controla o envio de dados entre os computadores (fim-a-fim)
- Objetivo: Evitar que o remetente envie mais dados do que o *buffer* do destinatário pode receber

Controle de Fluxo

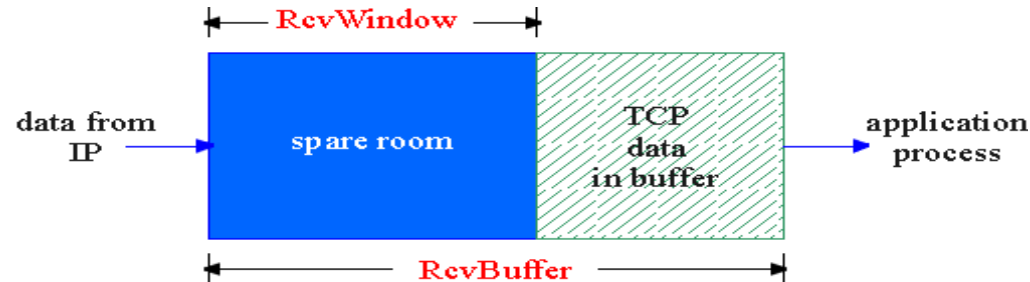


- Funcionamento:
 - Imaginemos um cenário onde o emissor A envia dados ao receptor B
 - Ao início da conexão, A informa para B o valor do *Buffer* que ele (A) possui disponível (Buffer de envio)
 - O receptor B então reserva um Buffer de recebimento do mesmo tamanho que o Buffer de envio de A

Controle de Fluxo



- Nesse cenário temos 2 questões diferentes:
 - A medida que os dados chegam, o **Buffer de recepção** vai se enchendo, formando o que é chamado de **Janela de recepção**
 - A Janela de recepção é a diferença entre o tamanho do Buffer de recepção e a quantidade de dados recebidos, mas ainda não retirados do Buffer pela aplicação

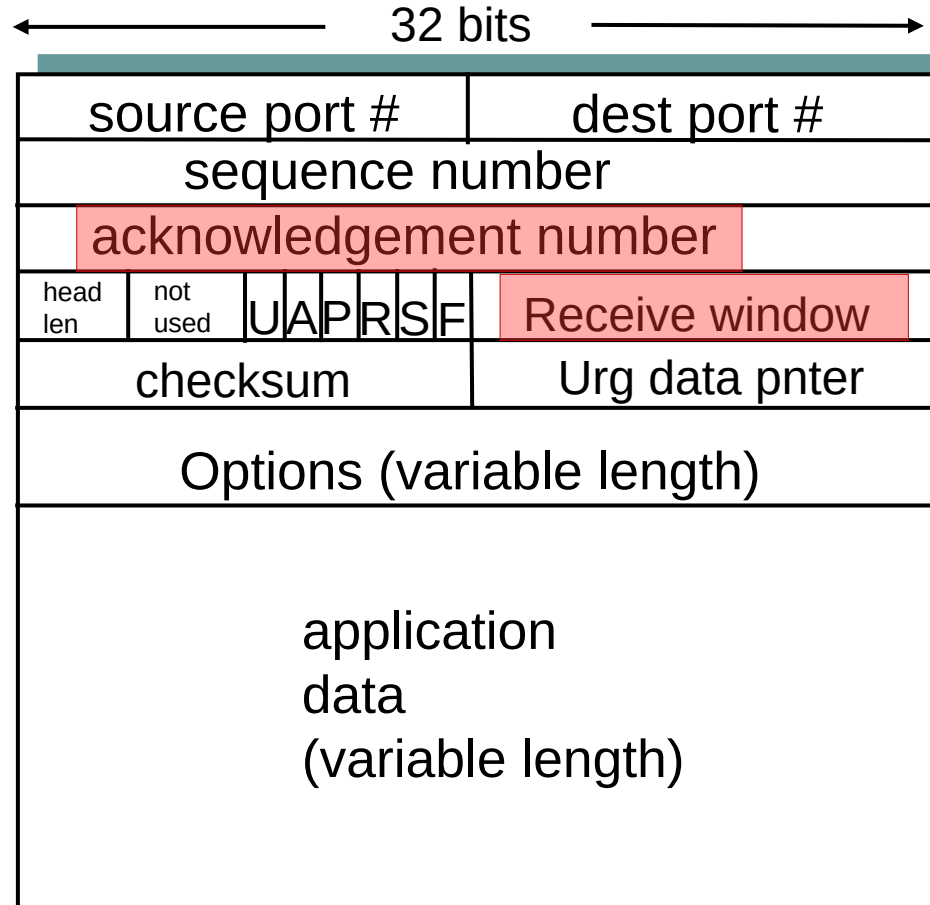




Controle de Fluxo

- À medida que o emissor envia dados e recebe ACKs do receptor, juntamente com os ACKs, o receptor envia também o tamanho da sua Janela de recepção
- Dessa forma, o emissor precisa controlar duas variáveis:
 - Último Byte Enviado
 - Último Byte Confirmado (ACKed)

Contrôle de Fluxo





Controle de Fluxo

- A diferença entre as variáveis do emissor informam a quantidade de Bytes “em trânsito” entre os computadores A e B (emissor e destinatário respectivamente)
- Logo:
Último Byte Enviado - Último Byte Confirmado $\leq$ Janela de Recepção



Controle de Fluxo

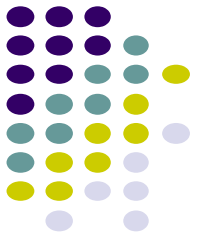
- Questões importantes a serem respondidas:
 - Como fica o Buffer do Emissor, caso receba um pacote que contém um novo número de ACK, contudo, um mesmo número para Receive Window?
- O que acontece se a Janela de recepção informada pelo Receptor chegar ao tamanho de 0?



Controle de Fluxo

- O que acontece se a Janela de recepção informada pelo Receptor chegar ao tamanho de 0?
 - Como o tamanho da Janela de recepção é informada nos ACKs, o Emissor continuará enviando pacotes contendo 1 Byte de dados. Mesmo com a chance alta do Byte ser descartado, é a forma de continuar “recebendo notícias” do Receptor

Controle de Fluxo



- Quando a Janela do Receptor aumentar novamente, o Emissor será informado através dos ACKs



Referências Bibliográficas

- KUROSE, J. F.; ROSS, K. W. **Redes de Computadores e a Internet: Uma abordagem Top-Down**. 6a ed., Pearson, 2013, São Paulo, SP.
- PETERSON L. L.; DAVIE B. S. **Redes de Computadores: uma Abordagem de Sistemas**. 3ª. Ed. São Paulo: Campus, 2004.